



ARTIFICIAL INTELLIGENCE

BSE (V-B)

SEMESTER PROJECT

TOPIC:

**“ECOMMERCE FRAUD DETECTION
SYSTEM”**

Submitted by:

Musfira Farooq(2023-BSE-045)
Shumaila Munsib(2023-BSE-062)
Zaheena Anwar(2023-BSE-070)

Submitted to:

Dr Irum Matloob

Table of Contents

1. Problem Description	5
Problem Definition.....	5
Input Data.....	5
Expected Output / Decision	5
2. Agentic AI System Design.....	5
Agent's Goals.....	5
Tools, Functions, or APIs	6
Machine Learning Frameworks	6
Data Preprocessing (Input Phase)	7
Feature Engineering (Behavioral Analysis).....	7
Model Inference (ANN Prediction)	7
Actions and Final Outcomes	8
3. Machine Learning Steps	8
Data Collection	8
Data Types and Memory Usage.....	9
Data Cleaning.....	10
Target Variable Analysis (Class Imbalance)	10
Feature Engineering / Feature Extraction	10
Feature–Target Separation	10
Feature Extraction	10
Categorical Feature Encoding.....	11
Data Type Normalization.....	11
Feature Scaling.....	12
Train–Test Split	12
Handling Class Imbalance Using SMOTE	13
Model Selection	13
Machine Learning Models	13
Logistic Regression.....	13
Decision Tree	13
K-Nearest Neighbors (KNN)	14

Artificial Neural Network (ANN) Development and Optimization	14
Artificial Neural Network (ANN).....	14
ANN Model Variants (Hyperparameter Tuning).....	14
ANN Model 1	15
ANN Model 2	16
ANN Model 3	17
Final ANN Model Selection	18
Training and Validation	19
Saving the Best Model	19
Model Deployment or Integration into the agentic system.....	20
ANN Prediction Agent.....	20
Behavioral Agent	20
Coordinator Agent	21
4. User Interface.....	21
Overview	21
Layout and Design	22
Front-End and Back-End Integration.....	24
User Experience	25
5. Conclusion	27
6. UI Screenshots	27

Table of Figures

Figure 1:Libraries installed	6
Figure 2:Data Collection.....	8
Figure 3:Dataset Preview	9
Figure 4&4.1:Identifying datatypes and nullvalues.....	9
Figure 5:Dropping column & Target column	10
Figure 6:Features extraction	11
Figure 7:Features Encoding	11
Figure 8:Datatype Normalisation.....	12
Figure 9:Standardization	12
Figure 10:Train &Test Split.....	13
Figure 11:Smote(Handles imbalance data).....	13
Figure 12 & 12.1:Models Training	14
Figure 13:ANN Hyperparameters Tunning	15
Figure 14 & 14.1: ANN model 1	16
Figure 15 &15.1: ANN model 2	17
Figure 16:ANN model 3	18
Figure 17:ANN model3 hyperparameter tuning	18
Figure 18:Evaluation Metrices of ANN Models.....	19
Figure 19:Models Comparison Table	19
Figure 20:Saving Best Model .pkl	20
Figure 21& 21.1 :GUI libraries and model loading.....	21
Figure 22:Header design	22
Figure 23:Left Panel(Input Section)	23
Figure 24:Right Panel(Output Section)	24
Figure 25:Actions Buttons	25
Figure 26:Prediction Logic	26
Figure 27:Behavioral Findings	26
Figure 28:Fraud Detected	27
Figure 29:Safe Transaction	28
Figure 30:Review Transaction.....	28

1. Problem Description

Problem Definition

Financial fraud is a growing challenge in modern digital banking systems due to the rapid increase in online transactions and evolving fraud techniques. Traditional manual and rule-based detection methods are ineffective at scale and fail to adapt to new fraud patterns, leading to missed fraudulent activities or false alarms. This project focuses on the automated detection of fraudulent financial transactions **using machine learning** and **agentic AI**, enabling the system to learn from historical transaction data, identify hidden fraud patterns, and make accurate real-time decisions with explainable reasoning.

Input Data

The fraud detection system processes structured transaction data, where each record represents a single financial transaction. The dataset includes numerical and categorical features such as transaction amount, type, timing and historical patterns. Before modeling, the data is cleaned, preprocessed, and transformed into numerical features to enable accurate detection of fraud and suspicious activity.

Expected Output / Decision

For each transaction, the system outputs:

- **Binary classification:** Indicates whether the transaction is **fraudulent** or **legitimate**.
- **Confidence score:** Represents the likelihood of fraud for the transaction.
- **Explainable reasoning:** Provided by the AI component, highlighting the key factors behind the decision to improve transparency and aid analysts.

These outputs support timely fraud prevention measures such as blocking suspicious transactions or alerting users minimizing incorrect detection and ensuring system reliability.

2. Agentic AI System Design

Agent's Goals

The primary goal of the system is to identify fraudulent financial transactions in real-time. The agents work collectively to:

- **Predict Fraud Risk:** Use a trained Artificial Neural Network (ANN) to analyze numerical and categorical transaction data.

- **Analyze Behavioral Patterns:** Assess user-specific history, such as account age, average transaction amount, and total transactions, to identify anomalies.
- **Coordinate Decision Making:** Aggregates outputs from the ANN and Behavioral Agent. Applies probability thresholding to make the final fraud decision.

Tools, Functions, or APIs

Machine Learning Frameworks

- **TensorFlow & Keras**

Used for building, training, and optimizing the Artificial Neural Network (ANN) fraud prediction model. Keras Sequential API is used to design multi-layer neural networks with dropout for regularization.

Data Processing Libraries

- **Pandas**

Used for loading transaction datasets, handling tabular data, and performing preprocessing operations such as column removal and feature selection.

- **NumPy**

Used for efficient numerical computations and array-based data transformations.

- **Scikit-learn**

Used for Data preprocessing (StandardScaler, LabelEncoder)

- **Train-test splitting**

Model evaluation (accuracy, precision, recall, F1-score) and Confusion matrix and classification report

- **Imbalanced Data Handling**

SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the dataset by oversampling the minority fraud class, improving the model's ability to detect fraudulent transactions.

```
import numpy as np
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import classification_report
```

Figure 1: Libraries installed

Model Persistence & Integration

- **Joblib**

Used to save and load preprocessing components such as:

- Feature scalers
- Label encoders
- Feature order

This ensures consistent preprocessing during real-time predictions in the user interface.

- **Saved ANN Model (.h5)**

The trained ANN model is saved and loaded during runtime to generate fraud predictions without retraining.

User Interface Framework

- **Tkinter (Python GUI Library)**

Used to develop a desktop-based graphical user interface that allows users to input transaction details and receive real-time fraud detection results.

- **Evaluation Tools**

- Confusion Matrix is used to visualize true positives, true negatives, false positives, and false negatives.
- Classification Report is used to compute Precision, Recall, F1-Score.

These metrics validate the effectiveness of the fraud detection system.

Step-by-Step Reasoning (Workflow)

The agents follow a structured logic to reach a decision:

Data Preprocessing (Input Phase)

The system first cleans the raw transaction data, handles missing values, and drops irrelevant identifiers like `transaction_id`.

Feature Engineering (Behavioral Analysis)

It extracts temporal features (hour, day, weekday) from the `transaction_time` and encodes categorical data like `merchant_category` and `country`.

Model Inference (ANN Prediction)

The preprocessed data is input into an Artificial Neural Network (ANN), where multiple dense layers learn complex, non-linear patterns to identify fraudulent transactions.

Probability Thresholding (Coordinator)

The coordinator agent evaluates the output of the sigmoid activation layer. If the probability is greater than 0.5, the transaction is flagged as fraud.

Actions and Final Outcomes

- **Transaction Scoring:**

The system calculates a fraud probability score for every incoming transaction.

- **Pattern Matching:**

It flags transactions that deviate significantly from a user's typical avg_amount_user or involve high-risk merchant_category types.

- **Resampling Action:**

During training, the system uses SMOTE to "balance" the data, ensuring the model doesn't ignore the rare fraud cases.

- **Final Outcome:**

The final output is a binary classification (0 for Legitimate, 1 for Fraud) accompanied by performance metrics that validate the system's accuracy and its ability to catch fraudulent activity (Recall).

3. Machine Learning Steps

Data Collection

The dataset was collected in CSV format and uploaded manually into Google Colab using the file upload utility. Once uploaded, the dataset was read into the system using the Pandas read_csv() function and stored in a DataFrame.

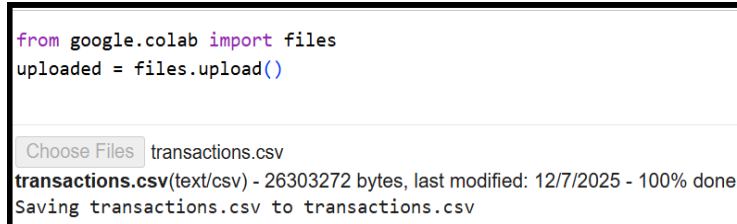


Figure 2: Data Collection

After loading, the dataset structure was verified using:

- df.head() to preview sample records
- df.shape to confirm dataset size

The dataset contains 299,695 rows (transactions), 17 columns (features)

```
import pandas as pd
csv_file = list(uploaded.keys())[0]
df = pd.read_csv(csv_file)

df.head(), df.shape
```

	transaction_id	user_id	account_age_days	total_transactions_user	
0	1	1	141	47	
1	2	1	141	47	
2	3	1	141	47	
3	4	1	141	47	
4	5	1	141	47	

	avg_amount_user	amount	country	bin_country	channel	merchant_category	
0	147.93	84.75	FR	FR	web	travel	
1	147.93	107.90	FR	FR	web	travel	
2	147.93	92.36	FR	FR	app	travel	
3	147.93	112.47	FR	FR	web	fashion	
4	147.93	132.91	FR	US	web	electronics	

	promo_used	avs_match	cvv_result	three_ds_flag	transaction_time	
0	0	1	1	1	2024-01-06T04:09:39Z	
1	0	0	0	0	2024-01-09T20:13:47Z	
2	1	1	1	1	2024-01-12T06:20:11Z	
3	0	1	1	1	2024-01-15T17:00:04Z	
4	0	1	1	1	2024-01-17T01:27:31Z	

	shipping_distance_km	is_fraud	
0	370.95	0	
1	149.62	0	
2	164.08	0	
3	397.40	0	
4	935.28	0	

(299695, 17))

Figure 3:Dataset Preview

To understand the dataset composition, the following checks were performed:

Data Types and Memory Usage

The df.info() function was used to inspect data types of all columns, non-null entries.

The dataset consists of:

- Integer features (IDs, flags, counts)
- Floating-point features (transaction amount, shipping distance)
- Categorical features (country, channel, merchant_category)
- Timestamp feature (transaction_time)

Missing Value Analysis

The df.isnull().sum() function was applied to identify missing values.

```
df.info()
df.isnull().sum()
```

```
RangeIndex: 299695 entries, 0 to 299694
Data columns (total 17 columns):
 #   Column                                  Non-Null Count  Dtype  
---  -
 0   transaction_id                         299695 non-null  int64  
 1   user_id                               299695 non-null  int64  
 2   account_age_days                       299695 non-null  int64  
 3   total_transactions_user                 299695 non-null  int64  
 4   avg_amount_user                         299695 non-null  float64 
 5   amount                                 299695 non-null  float64 
 6   country                                299695 non-null  object  
 7   bin_country                            299695 non-null  object  
 8   channel                                299695 non-null  object  
 9   merchant_category                       299695 non-null  object  
10   promo_used                             299695 non-null  int64  
11   avs_match                              299695 non-null  int64  
12   cvv_result                             299695 non-null  int64  
13   three_ds_flag                          299695 non-null  int64  
14   transaction_time                        299695 non-null  object  
15   shipping_distance_km                    299695 non-null  float64 
16   is_fraud                               299695 non-null  int64  
dtypes: float64(3), int64(9), object(5)
memory usage: 38.9+ MB
```

```
memory usage: 38.9+ MB
```

Column	Count
transaction_id	0
user_id	0
account_age_days	0
total_transactions_user	0
avg_amount_user	0
amount	0
country	0
bin_country	0
channel	0
merchant_category	0
promo_used	0
avs_match	0
cvv_result	0
three_ds_flag	0
transaction_time	0
shipping_distance_km	0
is_fraud	0

dtype: int64

Figure 4&4.1:Identifying datatypes and nullvalues

Data Cleaning

Removing Irrelevant Identifiers The following columns were removed:

- transaction_id
- user_id

These columns act only as unique identifiers and do not provide predictive value for fraud detection. Keeping them could introduce noise and bias into the model.

Target Variable Analysis (Class Imbalance)

The distribution of the target variable is_fraud was analyzed using value_counts().

Legitimate transactions (0): 293,083

Fraudulent transactions (1): 6,612

This revealed a severely imbalanced dataset, where fraudulent transactions represent a very small percentage of total records. Handling this imbalance was essential to avoid biased predictions.

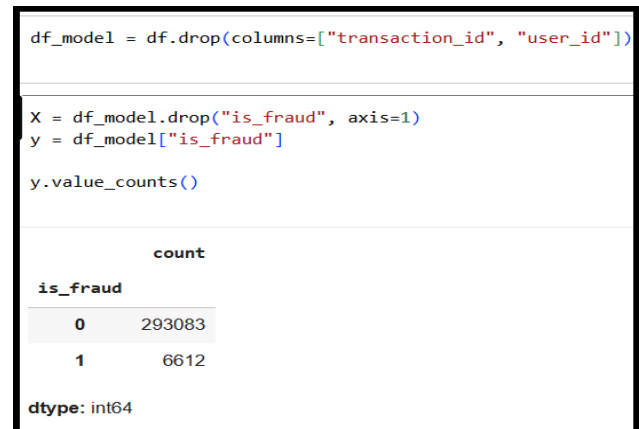


Figure 5: Dropping column & Target column

Feature Engineering / Feature Extraction

Feature–Target Separation

The dataset was split into:

- **Feature matrix (X)** → all columns except is_fraud
- **Target vector (y)** → is_fraud

This separation prepares the data for machine learning model training.

Feature Extraction

Datetime Conversion

The transaction_time column was converted into datetime format using pd.to_datetime() with UTC normalization to ensure consistency.

Feature Extraction from Time

From the converted timestamp, three new features were created:

- hour → Hour of transaction
- day → Day of the month
- weekday → Day of the week

These features help capture time-based fraud patterns, such as unusual transaction times. After extraction, the original transaction_time column was removed to prevent redundancy.

[8]
✓ 0s

```
X["transaction_time"] = pd.to_datetime(  
    X["transaction_time"],  
    errors="coerce",  
    utc=True  
)  
  
X["hour"] = X["transaction_time"].dt.hour  
X["day"] = X["transaction_time"].dt.day  
X["weekday"] = X["transaction_time"].dt.weekday  
  
X = X.drop("transaction_time", axis=1)
```

Figure 6: Features extraction

Categorical Feature Encoding

Several columns contained categorical (textual) values that machine learning models cannot process directly. This transformed categorical values into numerical representations.

For this reason:

- Label Encoding was applied to all object-type columns.
- A separate encoder was created and stored for each column.
- Encoders were saved to allow consistent encoding during deployment.

[9]
✓ 11s

```
from sklearn.preprocessing import LabelEncoder  
  
encoders = {}  
  
for col in X.columns:  
    if X[col].dtype == "object":  
        le = LabelEncoder()  
        X[col] = le.fit_transform(X[col].astype(str))  
        encoders[col] = le
```

Figure 7: Features Encoding

Data Type Normalization

All features were explicitly converted to numeric format using pd.to_numeric() to ensure compatibility with downstream preprocessing and model training steps.

```
X = X.apply(pd.to_numeric)
X.dtypes
```

...	0
account_age_days	int64
total_transactions_user	int64
avg_amount_user	float64
amount	float64
country	int64
bin_country	int64
channel	int64
merchant_category	int64
promo_used	int64
avs_match	int64
cvv_result	int64
three_ds_flag	int64
shipping_distance_km	float64
hour	int32
day	int32
weekday	int32

dtype: object

Figure 8: Datatype Normalisation

Feature Scaling

Standardization

The **StandardScaler** was applied to the feature matrix to:

- Center features around zero
- Scale features to unit variance

```
[11] ✓ 0s from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Figure 9: Standardization

The scaler was fitted only on the training data and later saved for deployment.

Train-Test Split

The dataset was split into: **80% training data** and **20% testing data**

Stratified sampling was used to preserve the original class distribution of fraud and non-fraud transactions in both subsets.

```
[12] ✓ 25s from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

Figure 10: Train & Test Split

Handling Class Imbalance Using SMOTE

SMOTE (Synthetic Minority Oversampling Technique) was applied only on the training data. Fraudulent samples were synthetically generated until both classes were balanced. This ensures the ANN learns fraud patterns effectively.

```
from imblearn.over_sampling import SMOTE

smote = SMOTE(
    sampling_strategy="auto",
    random_state=42
)

X_train_smote, y_train_smote = smote.fit_resample(
    X_train, y_train
)

print("Before SMOTE:", y_train.value_counts())
print("After SMOTE:", pd.Series(y_train_smote).value_counts())
```

```
Before SMOTE: is_fraud
0    234443
1     5313
Name: count, dtype: int64
After SMOTE: is_fraud
0    234443
1    234443
Name: count, dtype: int64
```

Figure 11: Smote (Handles imbalance data)

Model Selection

Machine Learning Models

Three traditional machine learning models were trained as baselines:

Logistic Regression

Used as a simple and interpretable baseline and maximum iterations increased to ensure convergence

Decision Tree

Captures non-linear relationships and provides rule-based decision making

K-Nearest Neighbors (KNN)

Uses distance-based classification and sensitive to feature scaling

Each model was trained and evaluated using:Accuracy,Precision,Recall,F1-Score

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import pandas as pd

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Define models
ml_models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "KNN": KNeighborsClassifier(n_neighbors=5)
}

results = []

for name, model in ml_models.items():
    print(f"\n★ Training {name}...")

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Print evaluation metrics
    print(f"Accuracy : {acc:.4f}")
    print(f"Precision: {prec:.4f}")
    print(f"Recall : {rec:.4f}")
    print(f"F1-Score : {f1:.4f}")

    # Store for final table
    results.append({
        "Model": name,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-Score": f1
    })
```

```
n_iter_i = _check_optimize_result(
Accuracy : 0.9855
Precision: 0.8396
Recall : 0.4111
F1-Score : 0.5519

★ Training Decision Tree...
Accuracy : 0.9864
Precision: 0.6743
Recall : 0.7236
F1-Score : 0.6981

★ Training KNN...
Accuracy : 0.9878
Precision: 0.8915
Recall : 0.4996
F1-Score : 0.6404
```

Figure 12 & 12.1:Models Training

Artificial Neural Network (ANN) Development and Optimization

Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) was selected due to its ability to learn complex, non-linear patterns, which are common in fraud detection problems. The baseline ANN architecture was designed to establish an initial performance benchmark using the preprocessed and SMOTE-balanced dataset. This model served as a reference point for further optimization through hyperparameter tuning.

ANN Model Variants (Hyperparameter Tuning)

After implementing the baseline ANN, hyperparameter tuning was performed to enhance model performance, particularly in improving fraud detection recall while maintaining high precision.

Hyperparameter tuning involved systematically experimenting with different configurations, including:

- **Learning rates** to control convergence behavior
- **Number of hidden layers and neurons** to capture deeper feature interactions
- **Dropout rates** to reduce overfitting
- **Number of epochs** to improve learning stability
- **Network depth and complexity**

Each unique configuration resulted in a separate ANN architecture. These models were labeled as **ANN Model 1**, **ANN Model 2**, and **ANN Model 3**.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

model = Sequential([
    Dense(64, activation="relu", input_shape=(X_train_smote.shape[1],)),
    Dropout(0.3),

    Dense(32, activation="relu"),
    Dropout(0.3),

    Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```
def build_ann(lr):
    model = Sequential([
        Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
        Dropout(0.3),
        Dense(32, activation='relu'),
        Dropout(0.2),
        Dense(1, activation='sigmoid')
    ])

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model
```

Figure 13: ANN Hyperparameters Tunning

ANN Model 1

ANN Model 1 was designed as the first tuned neural network, building upon the baseline architecture. It consists of:

- Input layer matching the number of processed features
- Hidden Layer 1: Dense layer with 64 neurons and ReLU activation function
- Hidden Layer 2: Dense layer with 32 neurons and ReLU activation function
- Dropout layers applied after hidden layers
- Output Layer: Single neuron and sigmoid activation for binary classification
- The model was trained on SMOTE-balanced training data to improve learning on fraudulent transactions.

Analysis

ANN Model 1 achieved **very high precision**, indicating that when the model predicts fraud, it is highly reliable.

```
model1 = build_ann(lr=0.001)

history1 = model1.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=20,
    batch_size=256,
    verbose=1
)
```

Figure 14 & 14.1: ANN model 1

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
750/750 ----- 5s 5ms/step - accuracy: 0.9185 - loss: 9.2889 - val_accuracy: 0.9823 - val_loss: 0.3003
Epoch 2/20
750/750 ----- 3s 3ms/step - accuracy: 0.9725 - loss: 0.4471 - val_accuracy: 0.9816 - val_loss: 0.0875
Epoch 3/20
750/750 ----- 3s 3ms/step - accuracy: 0.9807 - loss: 0.1329 - val_accuracy: 0.9813 - val_loss: 0.0794
Epoch 4/20
750/750 ----- 3s 3ms/step - accuracy: 0.9810 - loss: 0.1014 - val_accuracy: 0.9816 - val_loss: 0.0750
Epoch 5/20
750/750 ----- 3s 4ms/step - accuracy: 0.9808 - loss: 0.0915 - val_accuracy: 0.9817 - val_loss: 0.0723
Epoch 6/20
750/750 ----- 3s 4ms/step - accuracy: 0.9812 - loss: 0.0873 - val_accuracy: 0.9831 - val_loss: 0.0690
Epoch 7/20
750/750 ----- 3s 4ms/step - accuracy: 0.9817 - loss: 0.0819 - val_accuracy: 0.9831 - val_loss: 0.0672
Epoch 8/20
750/750 ----- 10s 10ms/step - accuracy: 0.9827 - loss: 0.0731 - val_accuracy: 0.9830 - val_loss: 0.0646
Epoch 9/20
750/750 ----- 6s 8ms/step - accuracy: 0.9829 - loss: 0.0724 - val_accuracy: 0.9841 - val_loss: 0.0617
Epoch 10/20
750/750 ----- 3s 3ms/step - accuracy: 0.9833 - loss: 0.0696 - val_accuracy: 0.9845 - val_loss: 0.0573
Epoch 11/20
750/750 ----- 4s 5ms/step - accuracy: 0.9842 - loss: 0.0645 - val_accuracy: 0.9842 - val_loss: 0.0563
Epoch 12/20
750/750 ----- 4s 3ms/step - accuracy: 0.9845 - loss: 0.0619 - val_accuracy: 0.9844 - val_loss: 0.0549
Epoch 13/20
750/750 ----- 3s 4ms/step - accuracy: 0.9844 - loss: 0.0606 - val_accuracy: 0.9858 - val_loss: 0.0504
Epoch 14/20
750/750 ----- 3s 4ms/step - accuracy: 0.9848 - loss: 0.0589 - val_accuracy: 0.9857 - val_loss: 0.0498
Epoch 15/20
750/750 ----- 4s 5ms/step - accuracy: 0.9859 - loss: 0.0551 - val_accuracy: 0.9863 - val_loss: 0.0494
Epoch 16/20
750/750 ----- 3s 4ms/step - accuracy: 0.9867 - loss: 0.0524 - val_accuracy: 0.9872 - val_loss: 0.0444
Epoch 17/20
750/750 ----- 3s 4ms/step - accuracy: 0.9865 - loss: 0.0511 - val_accuracy: 0.9882 - val_loss: 0.0435
Epoch 18/20
750/750 ----- 3s 3ms/step - accuracy: 0.9874 - loss: 0.0488 - val_accuracy: 0.9883 - val_loss: 0.0411
Epoch 19/20
750/750 ----- 3s 4ms/step - accuracy: 0.9874 - loss: 0.0491 - val_accuracy: 0.9896 - val_loss: 0.0403
Epoch 20/20
750/750 ----- 4s 3ms/step - accuracy: 0.9877 - loss: 0.0462 - val_accuracy: 0.9891 - val_loss: 0.0405
```

ANN Model 2

ANN Model 2 was designed to improve the balance between precision and recall by increasing model capacity.

- Input layer aligned with feature count
- Hidden Layer 1:Dense layer with 128 neurons and ReLU activation
- Hidden Layer 2:Dense layer with 64 neurons and ReLU activation
- Enhanced dropout regularization
- Output Layer: Single neuron and Sigmoid activation
- Training was performed on balanced data generated using SMOTE.

Analysis

ANN Model 2 achieved the **highest F1-score**, indicating the best trade-off between precision and recall. Although precision was slightly lower than ANN Model 1, recall improved significantly, which is critical in fraud detection where missing fraudulent cases is costly.

```

model2 = build_ann(lr=0.0005)

history2 = model2.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=128,
    verbose=1
)

```

```

... Epoch 2/30 ----- 5s 3ms/step - accuracy: 0.9784 - loss: 0.1525 - val_accuracy: 0.9816 - val_loss: 0.0826
Epoch 3/30 ----- 5s 3ms/step - accuracy: 0.9805 - loss: 0.0990 - val_accuracy: 0.9821 - val_loss: 0.0744
1499/1499 ----- 6s 4ms/step - accuracy: 0.9815 - loss: 0.0867 - val_accuracy: 0.9836 - val_loss: 0.0676
Epoch 4/30 ----- 11s 7ms/step - accuracy: 0.9821 - loss: 0.0794 - val_accuracy: 0.9836 - val_loss: 0.0633
1499/1499 ----- 6s 4ms/step - accuracy: 0.9826 - loss: 0.0711 - val_accuracy: 0.9847 - val_loss: 0.0603
Epoch 5/30 ----- 5s 3ms/step - accuracy: 0.9834 - loss: 0.0675 - val_accuracy: 0.9848 - val_loss: 0.0583
1499/1499 ----- 5s 3ms/step - accuracy: 0.9840 - loss: 0.0634 - val_accuracy: 0.9856 - val_loss: 0.0547
Epoch 6/30 ----- 5s 3ms/step - accuracy: 0.9838 - loss: 0.0612 - val_accuracy: 0.9858 - val_loss: 0.0522
1499/1499 ----- 6s 4ms/step - accuracy: 0.9853 - loss: 0.0565 - val_accuracy: 0.9862 - val_loss: 0.0509
Epoch 7/30 ----- 5s 3ms/step - accuracy: 0.9851 - loss: 0.0568 - val_accuracy: 0.9863 - val_loss: 0.0484
1499/1499 ----- 5s 3ms/step - accuracy: 0.9862 - loss: 0.0518 - val_accuracy: 0.9867 - val_loss: 0.0473
Epoch 8/30 ----- 6s 4ms/step - accuracy: 0.9865 - loss: 0.0507 - val_accuracy: 0.9878 - val_loss: 0.0438
1499/1499 ----- 5s 3ms/step - accuracy: 0.9876 - loss: 0.0468 - val_accuracy: 0.9877 - val_loss: 0.0436
Epoch 9/30 ----- 6s 4ms/step - accuracy: 0.9879 - loss: 0.0456 - val_accuracy: 0.9891 - val_loss: 0.0413
1499/1499 ----- 5s 3ms/step - accuracy: 0.9884 - loss: 0.0445 - val_accuracy: 0.9889 - val_loss: 0.0416
Epoch 10/30 ----- 4s 3ms/step - accuracy: 0.9883 - loss: 0.0437 - val_accuracy: 0.9893 - val_loss: 0.0395
1499/1499 ----- 5s 4ms/step - accuracy: 0.9890 - loss: 0.0421 - val_accuracy: 0.9897 - val_loss: 0.0398
Epoch 11/30 ----- 5s 3ms/step - accuracy: 0.9893 - loss: 0.0402 - val_accuracy: 0.9894 - val_loss: 0.0389
1499/1499 ----- 5s 3ms/step - accuracy: 0.9897 - loss: 0.0391 - val_accuracy: 0.9895 - val_loss: 0.0386
Epoch 12/30 ----- 5s 3ms/step - accuracy: 0.9895 - loss: 0.0396 - val_accuracy: 0.9898 - val_loss: 0.0382
1499/1499 ----- 5s 3ms/step - accuracy: 0.9897 - loss: 0.0394 - val_accuracy: 0.9887 - val_loss: 0.0403
Epoch 13/30 ----- 5s 4ms/step - accuracy: 0.9894 - loss: 0.0403 - val_accuracy: 0.9899 - val_loss: 0.0376
1499/1499 ----- 10s 3ms/step - accuracy: 0.9898 - loss: 0.0388 - val_accuracy: 0.9894 - val_loss: 0.0390
Epoch 14/30 ----- 5s 3ms/step - accuracy: 0.9898 - loss: 0.0392 - val_accuracy: 0.9899 - val_loss: 0.0388
1499/1499 ----- 5s 3ms/step - accuracy: 0.9899 - loss: 0.0391 - val_accuracy: 0.9901 - val_loss: 0.0369
Epoch 15/30 ----- 6s 4ms/step - accuracy: 0.9903 - loss: 0.0373 - val_accuracy: 0.9900 - val_loss: 0.0378
1499/1499 ----- 5s 3ms/step - accuracy: 0.9905 - loss: 0.0368 - val_accuracy: 0.9897 - val_loss: 0.0395
Epoch 16/30 ----- 5s 3ms/step - accuracy: 0.9902 - loss: 0.0377 - val_accuracy: 0.9903 - val_loss: 0.0389
1499/1499 ----- 6s 4ms/step - accuracy: 0.9899 - loss: 0.0386 - val_accuracy: 0.9899 - val_loss: 0.0379

```

Figure 15 & 15.1: ANN model 2

ANN Model 3

ANN Model 3 was developed to explore higher precision through stronger regularization.

- Input layer matching feature dimensions
- Deeper network structure with varied neuron counts
- ReLU activation functions
- Higher dropout rates to minimize overfitting
- Sigmoid-activated output layer

Analysis

ANN Model 3 achieved the **highest precision** among all ANN models, meaning it produced very few false positives. However, the lower recall reduced its effectiveness in detecting all fraudulent transactions, making it less suitable than ANN Model 2 for real-world deployment.

```
model3 = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.4),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

model3.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

history3 = model3.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=40,
    batch_size=128,
    verbose=1
)
```

Figure 16:ANN model 3

```
Epoch 20/40
1499/1499 — 5s 3ms/step - accuracy: 0.9894 - loss: 0.0408 - val_accuracy: 0.9901 - val_loss: 0.0389
Epoch 21/40
1499/1499 — 6s 4ms/step - accuracy: 0.9898 - loss: 0.0408 - val_accuracy: 0.9899 - val_loss: 0.0393
Epoch 22/40
1499/1499 — 5s 3ms/step - accuracy: 0.9900 - loss: 0.0402 - val_accuracy: 0.9898 - val_loss: 0.0384
Epoch 23/40
1499/1499 — 6s 4ms/step - accuracy: 0.9901 - loss: 0.0389 - val_accuracy: 0.9903 - val_loss: 0.0379
Epoch 24/40
1499/1499 — 5s 3ms/step - accuracy: 0.9898 - loss: 0.0404 - val_accuracy: 0.9898 - val_loss: 0.0391
Epoch 25/40
1499/1499 — 7s 4ms/step - accuracy: 0.9895 - loss: 0.0404 - val_accuracy: 0.9900 - val_loss: 0.0384
Epoch 26/40
1499/1499 — 5s 3ms/step - accuracy: 0.9899 - loss: 0.0400 - val_accuracy: 0.9900 - val_loss: 0.0386
Epoch 27/40
1499/1499 — 6s 4ms/step - accuracy: 0.9902 - loss: 0.0388 - val_accuracy: 0.9896 - val_loss: 0.0388
Epoch 28/40
1499/1499 — 5s 4ms/step - accuracy: 0.9900 - loss: 0.0385 - val_accuracy: 0.9892 - val_loss: 0.0403
Epoch 29/40
1499/1499 — 5s 4ms/step - accuracy: 0.9899 - loss: 0.0397 - val_accuracy: 0.9897 - val_loss: 0.0382
Epoch 30/40
1499/1499 — 6s 4ms/step - accuracy: 0.9898 - loss: 0.0395 - val_accuracy: 0.9900 - val_loss: 0.0386
Epoch 31/40
1499/1499 — 5s 4ms/step - accuracy: 0.9901 - loss: 0.0390 - val_accuracy: 0.9902 - val_loss: 0.0375
Epoch 32/40
1499/1499 — 6s 4ms/step - accuracy: 0.9901 - loss: 0.0384 - val_accuracy: 0.9886 - val_loss: 0.0393
Epoch 33/40
1499/1499 — 5s 4ms/step - accuracy: 0.9901 - loss: 0.0383 - val_accuracy: 0.9897 - val_loss: 0.0379
Epoch 34/40
1499/1499 — 10s 4ms/step - accuracy: 0.9904 - loss: 0.0380 - val_accuracy: 0.9897 - val_loss: 0.0385
Epoch 35/40
1499/1499 — 7s 4ms/step - accuracy: 0.9902 - loss: 0.0384 - val_accuracy: 0.9898 - val_loss: 0.0381
Epoch 36/40
1499/1499 — 9s 4ms/step - accuracy: 0.9903 - loss: 0.0375 - val_accuracy: 0.9897 - val_loss: 0.0383
Epoch 37/40
1499/1499 — 7s 4ms/step - accuracy: 0.9902 - loss: 0.0371 - val_accuracy: 0.9895 - val_loss: 0.0394
Epoch 38/40
1499/1499 — 6s 4ms/step - accuracy: 0.9906 - loss: 0.0370 - val_accuracy: 0.9892 - val_loss: 0.0389
Epoch 39/40
1499/1499 — 7s 4ms/step - accuracy: 0.9902 - loss: 0.0390 - val_accuracy: 0.9898 - val_loss: 0.0378
Epoch 40/40
1499/1499 — 5s 4ms/step - accuracy: 0.9905 - loss: 0.0374 - val_accuracy: 0.9899 - val_loss: 0.0395
```

Figure 17:ANN model3 hyperparameter tuning

Final ANN Model Selection

A comparative evaluation of all ANN architectures showed:

- **ANN Model 1:** Strong baseline with high precision
- **ANN Model 2:** Best overall balance and highest F1-score
- **ANN Model 3:** Highest precision but reduced recall

1874/1874					2s 1ms/step
ANN Model 1					
		precision	recall	f1-score	support
	0	0.99	1.00	0.99	58617
	1	0.94	0.58	0.72	1322
	accuracy			0.99	59939
	macro avg	0.97	0.79	0.86	59939
	weighted avg	0.99	0.99	0.99	59939
1874/1874					2s 1ms/step
ANN Model 2					
		precision	recall	f1-score	support
	0	0.99	1.00	1.00	58617
	1	0.92	0.62	0.74	1322
	accuracy			0.99	59939
	macro avg	0.96	0.81	0.87	59939
	weighted avg	0.99	0.99	0.99	59939
1874/1874					2s 1ms/step
ANN Model 3					
		precision	recall	f1-score	support
	0	0.99	1.00	1.00	58617
	1	0.95	0.62	0.75	1322
	accuracy			0.99	59939
	macro avg	0.97	0.81	0.87	59939
	weighted avg	0.99	0.99	0.99	59939

Figure 18:Evaluation Metrics of ANN Models

Training and Validation

All models (traditional + ANN) were compared using a final performance table. **ANN Model 2** achieved the highest indicating the best balance between precision and recall for fraud detection.

Final Model Comparison Table:					
	Model	Accuracy	Precision	Recall	F1-Score
0	ANN Model 2	0.990340	0.909091	0.615858	0.734282
1	ANN Model 1	0.990640	0.964736	0.589684	0.731964
2	ANN Model 3	0.990674	0.973146	0.585835	0.731379
3	Decision Tree	0.986369	0.673381	0.720554	0.696170
4	KNN	0.987838	0.891484	0.499615	0.640355
5	Logistic Regression	0.985535	0.839623	0.411085	0.551938

Figure 19:Models Comparison Table

Saving the Best Model

The selected ANN model was saved as:

- ann_fraud_model.h5

Preprocessing objects were saved as:

- ann_preprocessing.pkl,Scaler,Label encoders,Feature order

```
import joblib

# Assume 'model' is your trained model
joblib.dump(model, "best_model_selected_features.pkl")

['best_model_selected_features.pkl']

from google.colab import files

files.download("best_model_selected_features.pkl")
```

Figure 20: Saving Best Model .pkl

Model Deployment or Integration into the agentic system

The trained Artificial Neural Network (ANN) model was deployed within an Agentic Fraud Detection System designed to support modular, explainable, and real-time fraud analysis. The system consists of three specialized agents, each responsible for a distinct role in the fraud detection pipeline.

ANN Prediction Agent

The **ANN Prediction Agent** is responsible for core fraud classification. It loads the trained model (ann_fraud_model.h5). After that it applies all saved preprocessing steps such as Label encoding, Feature scaling, This agent ensures consistent inference by using the same preprocessing pipeline applied during training feature alignment. Receives new transaction data in real time and outputs a fraud probability score and binary prediction:

- 0 → Legitimate
- 1 → Fraudulent

Behavioral Agent

The **Behavioral Agent** analyzes transaction behavior beyond raw prediction. Evaluates user behavior patterns such as Transaction frequency, Amount deviations, Unusual transaction timing, Location or channel anomalies. It compares current transaction behavior with historical user patterns and assigns a behavioral risk score that complements the ANN prediction. This agent enhances fraud detection by identifying suspicious behavioral deviations that may not be captured by static features alone.

Coordinator Agent

The **Coordinator Agent** acts as the central decision-making and orchestration unit. It receives outputs from ANN Prediction Agent and Behavioral Agent. It aggregates risk scores and predictions, applies decision rules and thresholds, produces a final fraud decision with an explanation and logs decisions for audit and monitoring purpose. This agent ensures explainability, consistency and system-wide coordination.

Real-Time Processing and Explainability

The integrated agentic system enables real-time transaction analysis, modular agent communication and explainable fraud decisions

4. User Interface

Overview

The system features a graphical user interface (GUI) built using **CustomTkinter**, a modern and customizable extension of Tkinter. The interface allows users to interact seamlessly with the ANN-based Fraud Detection System and the intelligent agents, providing an intuitive and clear workflow for transaction analysis.

```
import customtkinter as ctk
import joblib
import pandas as pd
from tkinter import messagebox
from tensorflow.keras.models import load_model

from agentSystem import BehavioralRiskAgent, CoordinatorAgent
```

Figure 21& 21.1 :GUI libraries and model loading

```
# -----
# Constants
# -----
NUMERICAL_FEATURES = [
    "account_age_days",
    "avg_amount_user",
    "amount",
    "shipping_distance_km"
]

# -----
# Load ANN Model & Preprocessing
# -----
model = load_model("ann_fraud_model (5).h5")

preprocessor = joblib.load("ann_preprocessing (3).pkl")
scaler = preprocessor["scaler"]
encoders = preprocessor["encoders"]
feature_order = preprocessor["feature_order"]

# -----
# Initialize Agents
# -----
behavior_agent = BehavioralRiskAgent()
coordinator = CoordinatorAgent()

# -----
# GUI Setup
# -----
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("dark-blue")

app = ctk.CTk()
app.title("🕒 ANN Fraud Detection System")
app.state("zoomed")
```

Layout and Design

The interface is divided into **two main panels** within a clean and user-friendly window:

- **Header:**

Displays the system title and a brief description:

“ANN-Based Fraud Detection System – Artificial Neural Network • Intelligent Agents • Explainable AI”

The header uses a dark theme with a modern font and bright highlights for visual clarity.

```
# -----  
# Header  
# -----  
header = ctk.CTkFrame(app, height=90, fg_color=■ "#0f172a")  
header.pack(fill="x")  
  
ctk.CTkLabel(  
    header,  
    text="🌸 ANN-Based Fraud Detection System",  
    font=("Segoe UI", 32, "bold"),  
    text_color=■ "#38bdf8"  
).pack(pady=10)  
  
ctk.CTkLabel(  
    header,  
    text="Artificial Neural Network • Intelligent Agents • Explainable AI",  
    font=("Segoe UI", 14),  
    text_color=■ "#cbd5e1"  
).pack()  
  
# -----  
# Main Layout  
# -----  
main = ctk.CTkFrame(app, fg_color="transparent")  
main.pack(fill="both", expand=True, padx=30, pady=30)
```

Figure 22:Header design

- **Left Panel (Input Section):**

Provides input fields for all transaction-related features. Includes text entry boxes for numerical features and combo boxes for categorical features. It features “Analyze Transaction” button to trigger the backend prediction and agent reasoning and ensures input validation to prevent missing or incorrect entries.

```

# -----
# LEFT PANEL - INPUT
# -----
left = ctk.CTkScrollableFrame(main, width=480, fg_color=█ "#020617", corner_radius=18)
left.pack(side="left", fill="y", padx=15)

ctk.CTkLabel(
    left,
    text="Transaction Details",
    font=("Segoe UI", 22, "bold"),
    text_color=█ "#e5e7eb"
).pack(pady=20)

entries = {}

def add_label(text):
    ctk.CTkLabel(
        left,
        text=text,
        font=("Segoe UI", 13),
        text_color=█ "#e5e7eb"
    ).pack(anchor="w", pady=(14, 4))

for feature in feature_order:
    add_label(feature.replace("_", " ").title())

    if feature in encoders:
        cb = ctk.CTkComboBox(
            left,
            values=list(encoders[feature].classes_),
            width=420
        )
        cb.pack()
        entries[feature] = cb
    else:
        en = ctk.CTkEntry(left, width=420, placeholder_text="Enter value")
        en.pack()
        entries[feature] = en

ctk.CTkButton(
    left,
    text="🔍 Analyze Transaction",
    height=55,
    width=420,
    font=("Segoe UI", 18, "bold"),
    fg_color=█ "#2563eb",
    hover_color=█ "#1e40af",
    command=lambda: predict()
).pack(pady=30)

```

Figure 23: Left Panel (Input Section)

- **Right Panel (Output Section):**

Displays the transaction report, including ANN fraud probability, behavioral agent findings, coordinator agent's final decision and recommended actions. It provides decision labels with clear visual indicators for:

- **SAFE** transactions (green)
- **FRAUD** transactions (red)
- **REVIEW REQUIRED** transactions (yellow)

It includes action buttons for the user to approve, reject, or review transactions based on the system's recommendations.

```
# -----  
# RIGHT PANEL - RESULT  
# -----  
right = ctk.CTkFrame(main, fg_color=■ "#020617", corner_radius=18)  
right.pack(side="right", fill="both", expand=True, padx=15)  
  
ctk.CTkLabel(  
    right,  
    text="Transaction Report",  
    font=("Segoe UI", 22, "bold"),  
    text_color=□ "#e5e7eb"  
).pack(pady=20)  
  
decision_label = ctk.CTkLabel(  
    right,  
    text="Awaiting Analysis",  
    font=("Segoe UI", 28, "bold"),  
    text_color=■ "#38bdf8"  
)  
decision_label.pack(pady=10)  
  
result_box = ctk.CTkTextbox(  
    right,  
    font=("Consolas", 14),  
    corner_radius=12,  
    fg_color=■ "#020617",  
    text_color=□ "#e5e7eb",  
    height=360  
)  
result_box.pack(padx=25, pady=10, fill="both", expand=True)  
result_box.insert("end", "Run analysis to view transaction report.")  
result_box.configure(state="disabled")
```

Figure 24:Right Panel(Output Section)

Front-End and Back-End Integration

The interface is fully integrated with the system's backend, demonstrating the following:

Machine Learning Model:

Loads a pre-trained **Artificial Neural Network (ANN)** model for fraud prediction and preprocesses user inputs using scaler and encoder objects to ensure consistency with the model's training data.

Agent-Based Reasoning:

The Behavioral Risk Agent analyzes transaction patterns and highlights suspicious behaviors. The CoordinatorAgent combines the ANN prediction with behavioral analysis to produce a final decision.

Dynamic Display:

Updates the report textbox and decision label dynamically based on the prediction and agent reasoning and displays contextual explanations, behavioral findings, and recommended actions for transparency.

User Experience

The interface features a **dark theme** with contrasting highlights for readability. Input fields are clearly labeled following consistent formatting. The transaction results are displayed in a structured and readable format ensuring that users can easily interpret predictions and agent recommendations. Buttons are context-sensitive appearing only when relevant, reducing clutter and guiding the user's actions.

```
# -----
# ACTION BUTTONS
# -----
action_frame = ctk.CTkFrame(right, fg_color="transparent")
action_frame.pack(pady=20)

approve_btn = ctk.CTkButton(
    action_frame,
    text="✅ APPROVE TRANSACTION",
    fg_color="#22c55e",
    hover_color="#16a34a",
    font=("Segoe UI", 16, "bold"),
    command=lambda: messagebox.showinfo("Approved", "Transaction Approved Successfully")
)

reject_btn = ctk.CTkButton(
    action_frame,
    text="❌ REJECT TRANSACTION",
    fg_color="#ef4444",
    hover_color="#b91c1c",
    font=("Segoe UI", 16, "bold"),
    command=lambda: messagebox.showwarning("Rejected", "Transaction Rejected")
)

review_approve_btn = ctk.CTkButton(
    action_frame,
    text="✅ APPROVE AFTER REVIEW",
    fg_color="#22c55e",
    hover_color="#16a34a",
    font=("Segoe UI", 16, "bold"),
    command=lambda: messagebox.showinfo("Approved", "Transaction Approved After Review")
)

review_reject_btn = ctk.CTkButton(
    action_frame,
    text="❌ REJECT AFTER REVIEW",
    fg_color="#ef4444",
    hover_color="#b91c1c",
    font=("Segoe UI", 16, "bold"),
    command=lambda: messagebox.showwarning("Rejected", "Transaction Rejected After Review")
)

def clear_action_buttons():
    for widget in action_frame.winfo_children():
        widget.pack_forget()
```

Figure 25: Actions Buttons

```

# -----
# Prediction Logic
# -----
def predict():
    try:
        clear_action_buttons()

        input_data = {}
        for feature in feature_order:
            value = entries[feature].get()
            if value == "":
                raise ValueError(f"{feature} cannot be empty")

            if feature in encoders:
                input_data[feature] = encoders[feature].transform([value])[0]
            else:
                input_data[feature] = float(value)

        df = pd.DataFrame([input_data])
        df = df[feature_order]
        df[NUMERICAL_FEATURES] = scaler.transform(df[NUMERICAL_FEATURES])

        fraud_probability = model.predict(df.values)[0][0]

        behavior_result = behavior_agent.analyze(input_data)
        final_result = coordinator.decide(
            ann_probability=fraud_probability,
            behavioral_result=behavior_result
        )

        if final_result["final_decision"] == "FRAUD":
            decision_label.configure(text="🚨 FRAUD DETECTED", text_color="🔴#ef4444")
            reject_btn.pack(padx=10, pady=5)

        elif final_result["final_decision"] == "SAFE":
            decision_label.configure(text="✅ SAFE TRANSACTION", text_color="🟢#22c55e")
            approve_btn.pack(padx=10, pady=5)

        else:
            decision_label.configure(text="⚠️ REVIEW REQUIRED", text_color="🟡#f59e0b")
            review_approve_btn.pack(side="left", padx=10)
            review_reject_btn.pack(side="left", padx=10)

        report = f"""
ANN Fraud Probability: {fraud_probability:.2f}

Final Decision: {final_result['final_decision']}

Reason:
{final_result['reason']}
"""

```

Figure 26: Prediction Logic

```

Behavioral Findings:
- {'; '.join(final_result['behavior_reasons'])}

Recommended Action:
{final_result['system_action']}
"""

    result_box.configure(state="normal")
    result_box.delete("1.0", "end")
    result_box.insert("end", report)
    result_box.configure(state="disabled")

except Exception as e:
    decision_label.configure(text="❌ ERROR", text_color="🔴#ef4444")
    result_box.configure(state="normal")
    result_box.delete("1.0", "end")
    result_box.insert("end", str(e))
    result_box.configure(state="disabled")

# -----
# Run App
# -----
app.mainloop()

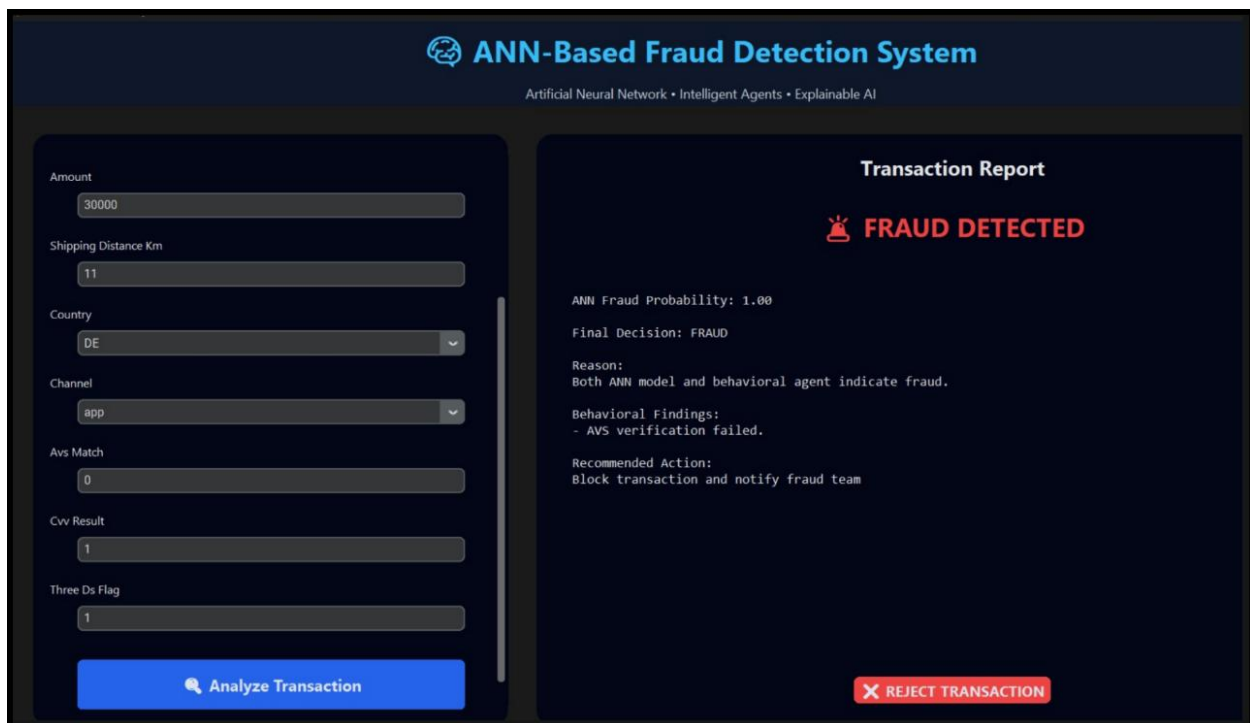
```

Figure 27: Behavioral Findings

5. Conclusion

The ANN-Based Fraud Detection System with Intelligent Agents is a deployment ready solution that combines machine learning, behavioral analysis, and agent-based reasoning to provide accurate, explainable and actionable fraud detection. Its user-friendly interface allows seamless input of transaction data and presents clear, structured reports with real-time decisions and recommended actions. By integrating the ANN model with behavioral and coordinator agents, the system ensures transparent reasoning enabling users to understand the factors behind each classification. Its modular architecture and use of standard Python frameworks make it scalable, maintainable and suitable for real-world deployment offering financial institutions an effective tool to enhance transaction security and reduce fraud-related losses.

6. UI Screenshots



The screenshot displays the user interface of the ANN-Based Fraud Detection System. The interface is divided into two main sections: a form for inputting transaction details on the left and a transaction report on the right.

Transaction Input Form (Left):

- Amount:** 30000
- Shipping Distance Km:** 11
- Country:** DE
- Channel:** app
- Avs Match:** 0
- Cvv Result:** 1
- Three Ds Flag:** 1
- Button:** Analyze Transaction

Transaction Report (Right):

- Header:** Transaction Report
- Status:** FRAUD DETECTED (indicated by a red flame icon)
- ANN Fraud Probability:** 1.00
- Final Decision:** FRAUD
- Reason:** Both ANN model and behavioral agent indicate fraud.
- Behavioral Findings:**
 - AVS verification failed.
- Recommended Action:** Block transaction and notify fraud team
- Button:** REJECT TRANSACTION

Figure 28:Fraud Detected

30000

Shipping Distance Km

11

Country

DE

Channel

app

Avs Match

1

Cvv Result

1

Three Ds Flag

1

Analyze Transaction

Transaction Report

SAFE TRANSACTION

ANN Fraud Probability: 0.00

Final Decision: SAFE

Reason:
Both agents agree transaction is safe.

Behavioral Findings:
- No abnormal behavioral patterns detected.

Recommended Action:
Approve transaction

APPROVE TRANSACTION

Figure 29:Safe Transaction

30000

Shipping Distance Km

11

Country

DE

Channel

app

Avs Match

1

Cvv Result

1

Three Ds Flag

1

Analyze Transaction

Transaction Report

REVIEW REQUIRED

ANN Fraud Probability: 1.00

Final Decision: REVIEW

Reason:
ANN detected fraud risk but behavior appears normal.

Behavioral Findings:
- No abnormal behavioral patterns detected.

Recommended Action:
Send transaction for manual review

APPROVE AFTER REVIEW

REJECT AFTER REVIEW

Figure 30:Review Transaction